

Representation and Time–Memory Tradeoffs in Symmetric Cryptography: Finite-Field Basis Transformations for AES and Reproducible Hellman Experiments

Ali Mkhida
Algorizk Labs

Current draft maintainer

Originating from an M2P SCCI master’s research project at Ensimag–UJF
developed with Abdourahmane Sakho and Maad El Yadari

Working ePrint draft — 28 August 2026

Abstract

This note upgrades a historical master’s research project on high-performance cryptography into a reproducible research artifact. The original presentation combined finite-field representation changes for AES-like arithmetic with practical secret-key recovery experiments based on exhaustive search, Hellman time–memory tradeoffs, and distinguished points. The repository now preserves the original presentation as a historical PDF, while this note gives a separate ePrint-style reconstruction.

The first part reconstructs the representation mathematics directly from the preserved slides: the concrete $\mathbb{F}(16)$ normal basis, its exact bilinear multiplication formula, the squaring permutation, the displayed 8-by-8 isomorphism matrices, and the two-level composite-field non-LUT inversion proposal. The second part gives a modern reproducible implementation of the cryptanalytic experiments over deliberately reduced DES state spaces. The main empirical observation is that, for the tested single-map Hellman construction with $mt/N = 1$, the median exact table coverage is only about 16.8%, far below the independent-sample occupancy reference $1 - e^{-1} \simeq 63.2\%$. The gap is explained by functional-graph coalescence: chain states are not independent samples but iterates of one fixed map.

1 Historical artifact and updated goal

The original project was titled *High Performance Computing for Cryptography*. That title correctly described the original research context, but it does not reveal the mathematical object studied by the modernized repository. The updated repository is organized around two related themes:

1. finite-field representation changes for efficient AES-like arithmetic;
2. cryptanalytic time–memory tradeoffs for reduced key spaces.

The historical presentation remains preserved unchanged in the repository at

`docs/original-presentation.pdf`.

This document is separate: it is the research-note version, intended to evolve toward an ePrint-style manuscript. Where the original slides were visual and compressed, this note makes the algebraic objects explicit and separates historical claims from newly reproduced measurements.

The reconstruction below is based directly on slides 4–19 of the preserved presentation [8]. Slides 20 and 21 contain only the phrase “PROOF ON BOARD” for, respectively, secret-key recovery and the claimed 200 MHz AES throughput. Consequently, this note does not reconstruct or endorse a proof or a 200 MHz implementation result from those two slides; such claims require independent source material or a new implementation.

2 Finite fields in characteristic two

Let $n \geq 1$. The binary extension field $\mathbb{F}(2^n)$ may be represented as

$$\mathbb{F}(2^n) \simeq \mathbb{F}(2)[X]/(p(X)),$$

where p is an irreducible polynomial of degree n . If α denotes the class of X , the *polynomial basis* is

$$(1, \alpha, \alpha^2, \dots, \alpha^{n-1}).$$

Every field element is written uniquely as

$$a = a_0 + a_1\alpha + \dots + a_{n-1}\alpha^{n-1}, \quad a_i \in \mathbb{F}(2).$$

A second important representation is a *normal basis*. An element $\beta \in \mathbb{F}(2^n)$ is normal over $\mathbb{F}(2)$ if

$$\mathcal{N}(\beta) = (\beta, \beta^2, \beta^{2^2}, \dots, \beta^{2^{n-1}})$$

is a basis of $\mathbb{F}(2^n)$ over $\mathbb{F}(2)$. In this representation, the Frobenius map

$$\varphi : x \mapsto x^2$$

acts as a cyclic shift on coordinates:

$$\sum_{i=0}^{n-1} b_i \beta^{2^i} \mapsto \sum_{i=0}^{n-1} b_i \beta^{2^{i+1}}.$$

This is the central reason normal bases are attractive in hardware: squaring can be implemented by rewiring rather than by a general multiplier.

3 The historical $\mathbb{F}(16)$ model

The historical presentation does not work only at the abstract $\mathbb{F}(2^n)$ level. It fixes the concrete quartic field

$$K_4 = \mathbb{F}(2)[X]/(X^4 + X^3 + X^2 + X + 1)$$

and lets α denote the residue class of X . The slides first use the ordered polynomial (“canonical”) basis

$$\mathcal{B}_{\text{can}} = (\alpha^3, \alpha^2, \alpha, 1).$$

Because

$$\alpha^4 + \alpha^3 + \alpha^2 + \alpha + 1 = 0,$$

we have $\alpha^5 = 1$ and $\alpha \neq 1$.

The presentation then uses the ordered normal basis

$$\mathcal{B}_{\text{nor}} = (\alpha^8, \alpha^4, \alpha^2, \alpha).$$

The displayed identities are

$$1_{K_4} = \alpha^8 + \alpha^4 + \alpha^2 + \alpha,$$

together with

$$\begin{aligned} \alpha^3 &= \alpha^8, & \alpha^5 &= \alpha^{10} = \alpha^8 + \alpha^4 + \alpha^2 + \alpha, \\ \alpha^6 &= \alpha^{16} = \alpha, & \alpha^9 &= \alpha^4, & \alpha^{12} &= \alpha^2. \end{aligned}$$

These are consistent with $\alpha^5 = 1$. In particular, Frobenius squaring cycles the basis elements

$$\alpha^8 \mapsto \alpha, \quad \alpha^4 \mapsto \alpha^8, \quad \alpha^2 \mapsto \alpha^4, \quad \alpha \mapsto \alpha^2.$$

For coordinate vectors

$$x = [a_3, a_2, a_1, a_0]_{\mathcal{B}_{\text{nor}}}, \quad y = [b_3, b_2, b_1, b_0]_{\mathcal{B}_{\text{nor}}},$$

this means

$$x = a_3\alpha^8 + a_2\alpha^4 + a_1\alpha^2 + a_0\alpha,$$

and similarly for y .

3.1 Exact multiplication formula from the slides

Slide 7 expands the product in the normal basis. Defining the common term

$$q = a_0b_2 \oplus a_2b_0 \oplus a_1b_3 \oplus a_3b_1,$$

the four output coordinates can be written compactly as

$$\begin{aligned} c_3 &= a_0b_1 \oplus a_1b_0 \oplus a_2b_2 \oplus q, \\ c_2 &= a_0b_3 \oplus a_3b_0 \oplus a_1b_1 \oplus q, \\ c_1 &= a_2b_3 \oplus a_3b_2 \oplus a_0b_0 \oplus q, \\ c_0 &= a_1b_2 \oplus a_2b_1 \oplus a_3b_3 \oplus q. \end{aligned}$$

Thus

$$xy = c_3\alpha^8 + c_2\alpha^4 + c_1\alpha^2 + c_0\alpha.$$

The repeated term q is precisely the common subexpression highlighted in the historical slide. Sharing it is a direct logic optimization: the same XOR subnetwork feeds all four output coordinates.

The repository now includes a small verifier that exhaustively checks this formula on all 16^2 input pairs against multiplication modulo $X^4 + X^3 + X^2 + X + 1$.

3.2 Squaring is a coordinate permutation

Setting $y = x$ in the displayed multiplication formula, or simply applying the Frobenius map, gives the historical slide-8 identity

$$x^2 = a_2\alpha^8 + a_1\alpha^4 + a_0\alpha^2 + a_3\alpha.$$

Hence

$$[a_3, a_2, a_1, a_0] \mapsto [a_2, a_1, a_0, a_3].$$

No field multiplier is required for this operation in the chosen normal basis: squaring is a cyclic coordinate permutation. This is the precise content of the slide’s statement that squaring “comes for free in hardware.” The same normal-basis principle underlies inversion algorithms such as Itoh–Tsujii [7].

4 The tower-field isomorphism used for the S-box

AES represents a byte as an element of

$$K_8 = \mathbb{F}(2)[T]/(T^8 + T^4 + T^3 + T + 1),$$

the standard AES field [3]. The historical presentation writes the S-box schematically as

$$S(t) = A_s t^{-1} + B_s,$$

where the subscript here only disambiguates the affine-map constants from the quadratic-extension coefficients used below.

The central proposal in slide 5 is to choose a field isomorphism

$$\delta : K_8 \longrightarrow K_4[Y]/(Y^2 + A_q Y + B_q)$$

and perform the multiplicative inversion in the quadratic extension of K_4 :

$$t \xrightarrow{\delta} a_1 Y + a_0 \xrightarrow{\text{inverse}} b_1 Y + b_0 \xrightarrow{\delta^{-1}} t^{-1}.$$

This is the exact representation-level decomposition shown in the presentation. It is closely related to the composite-field AES S-box architectures studied by Satoh et al. [6] and Canright [5].

4.1 Quadratic-extension inversion

Let

$$z = a_1 Y + a_0, \quad Y^2 + A_q Y + B_q = 0.$$

The conjugate root of Y is $Y + A_q$, so

$$\bar{z} = a_1 Y + (a_0 + A_q a_1).$$

Its norm down to K_4 is

$$N(z) = z\bar{z} = a_0^2 + A_q a_0 a_1 + B_q a_1^2.$$

For $z \neq 0$,

$$z^{-1} = \frac{a_1 Y + (a_0 + A_q a_1)}{a_0^2 + A_q a_0 a_1 + B_q a_1^2}.$$

The normal basis from the previous section makes both squarings in the denominator coordinate permutations. This explains why the historical project first optimized K_4 arithmetic and only then returned to the 8-bit S-box.

5 Constructing and representing the isomorphism

The slides describe the isomorphism by matching generators. Let $K_8^\times$ and

$$L^\times = (K_4[Y]/(Y^2 + A_q Y + B_q))^\times$$

be the multiplicative groups. If $\alpha_8 \in K_8^\times$ and $\beta \in L^\times$ are roots of the same irreducible degree-8 polynomial P , define

$$\delta(\alpha_8) = \beta, \quad \delta(\alpha_8^i) = \beta^i.$$

Because

$$|K_8^\times| = 2^8 - 1 = 255 = 3 \cdot 5 \cdot 17,$$

a candidate g is primitive exactly when

$$g^{255/3} \neq 1, \quad g^{255/5} \neq 1, \quad g^{255/17} \neq 1.$$

This is the standard precise generator test corresponding to the abbreviated criterion written on slide 11.

The historical slide states the candidate

$$\beta = [1, 0, 0, 1] Y + [0, 0, 0, 0]$$

under its K_4 coordinate convention and records

$$P(\beta) = 0, \quad P(T) = T^8 + T^4 + T^3 + T + 1.$$

It then defines a derived map δ_2 by its image of the polynomial generator and extends it multiplicatively:

$$\delta_2(T^i) = \delta_2(T)^i.$$

The target coefficients A_q, B_q are not written on the surviving slides, so the root calculation itself cannot be independently reconstructed from the PDF alone. What *is* recoverable exactly is the resulting binary linear map.

5.1 Exact 8×8 matrices

Slide 12 gives the matrix of δ_2 and its inverse under the bit ordering used in the presentation:

$$D = \begin{bmatrix} 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix},$$

and

$$D^{-1} = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 1 & 0 & 0 \end{bmatrix}.$$

The modernization verifier checks over $\mathbb{F}(2)$ that

$$DD^{-1} = D^{-1}D = I_8.$$

6 Hardware meaning of the isomorphism matrices

For an 8-bit coordinate vector v , the basis conversion is simply

$$\delta_2(v) = Dv, \quad \delta_2^{-1}(v) = D^{-1}v \quad \text{over } \mathbb{F}(2).$$

Each output bit is therefore an XOR of selected input bits, and all eight output coordinates can be computed in parallel. This is exactly the point made in slides 14–15.

A natural structural cost is the matrix Hamming weight

$$\text{wt}(M) = |\{(i, j) : M_{ij} = 1\}|.$$

For the matrices printed in the historical presentation,

$$\text{wt}(D) = 30, \quad \text{wt}(D^{-1}) = 32.$$

The slide explicitly suggests searching for an isomorphism with low Hamming weight. In modern terms, one should optimize not only total ones, but also shared XOR subexpressions, XOR depth, fanout, and placement/routing delay. Canright’s later systematic treatment of basis choices and isomorphism matrices is directly relevant to this optimization viewpoint [5].

7 A second composite-field layer and the non-LUT proposal

Slides 17–19 propose a second decomposition to shorten the longest combinational path. The historical notation uses maps δ_a and δ_b and states the goal of factoring the first isomorphism through a smaller field. To avoid the composition-order ambiguity in the slide notation, we write the two operational stages as

$$\Delta_8 : K_8 \longrightarrow K_4[Y]/(Y^2 + A_qY + B_q)$$

and

$$\Delta_4 : K_4 \longrightarrow \mathbb{F}(4)[Z]/(Z^2 + CZ + D_q).$$

Extend Δ_4 coefficientwise to the quadratic extension. The combined representation change is then unambiguously

$$\Psi = \hat{\Delta}_4 \circ \Delta_8.$$

The pipeline corresponding to slide 19 is

$$t \xrightarrow{\Delta_8} a_1Y + a_0 \xrightarrow{\hat{\Delta}_4} \tilde{a} \xrightarrow{\text{inverse}} \tilde{c} \xrightarrow{\hat{\Delta}_4^{-1}} (a_1Y + a_0)^{-1} \xrightarrow{\Delta_8^{-1}} t^{-1}.$$

The key arithmetic observation is that for every nonzero $\gamma \in \mathbb{F}(4)$,

$$\gamma^3 = 1 \quad \implies \quad \gamma^{-1} = \gamma^2.$$

Thus inversion in $\mathbb{F}(4)$ is just squaring on nonzero elements. In a normal basis, that squaring is again a coordinate permutation. This is the precise mathematical interpretation of the historical statement that the smallest inversion stage “comes for free in hardware,” and it explains the proposed non-LUT S-box datapath.

The historical slides do not provide the actual C, D_q , the concrete Δ_4 matrix, a synthesized critical path, or an implementation report. Those are therefore future-work items rather than reconstructed results.

8 From representation arithmetic to cryptanalytic experiments

The finite-field part of the project concerns the cost of one encryption datapath. The cryptanalytic part concerns the cost of searching over many candidate keys. These are linked but distinct:

- finite-field basis choices change the cost of encryption rounds;
- time–memory tradeoffs change the cost profile of key recovery.

The original master’s research project connected these views by considering secret-key recovery experiments after the AES arithmetic discussion. The modern repository makes the second part reproducible over deliberately reduced DES state spaces. DES is obsolete and is used here only because it provides a small, standard block cipher setting suitable for reproducible experiments.

9 Reduced DES state model

The historical Java prototype varied four raw DES bytes. However, a DES key is encoded in eight bytes with one parity bit per byte. Thus four varying raw bytes do not automatically give 32 independent effective DES key bits.

The modern model defines a reduced state space explicitly:

$$X_b = \{0, 1, \dots, 2^b - 1\}, \quad 1 \leq b \leq 28.$$

A state $x \in X_b$ is mapped to a canonical DES key encoding $K(x)$ with odd parity. The first four DES bytes are fixed, and the final four bytes carry up to seven effective bits each. This gives at most 28 effective variable bits.

For a fixed plaintext P , the encryption oracle is

$$C_x = \text{Enc}_{K(x)}(P).$$

A reduction function

$$R : \{0, 1\}^* \rightarrow X_b$$

maps ciphertexts back into reduced states. The core functional graph is

$$F(x) = R(\text{Enc}_{K(x)}(P)).$$

Hellman chains and distinguished-point chains are both walks in this directed functional graph.

10 Exhaustive search

Given a target ciphertext C^* , exhaustive search tests every state:

$$\text{Enc}_{K(x)}(P) \stackrel{?}{=} C^*.$$

It requires no offline table and no stored endpoints. Its memory cost is constant, but its online cost is $O(2^b)$ encryptions in the worst case.

In the verified experiment with $b = 16$, exhaustive search recovers all targets because it covers the entire reduced state space by definition.

11 Hellman chains

A Hellman table [1] is generated by choosing start states

$$s_1, \dots, s_m \in X_b$$

and iterating F for a fixed chain length t :

$$s_i, F(s_i), F^2(s_i), \dots, F^t(s_i).$$

Only the pair

$$(s_i, F^t(s_i))$$

is stored. The second component is the endpoint.

For a target ciphertext C^* , the online phase computes

$$R(C^*), F(R(C^*)), F^2(R(C^*)), \dots$$

and checks whether one of these projected endpoints appears in the table. When an endpoint matches, the corresponding stored chain is regenerated from its start point and the target ciphertext is verified directly.

The modern implementation stores endpoints in a hash index rather than repeatedly scanning every endpoint. It also retains endpoint collisions and regenerates all candidate chains, because different starts may lead to the same endpoint.

11.1 Why the independent occupancy heuristic is optimistic

If mt states were sampled independently and uniformly from a universe of size $N = 2^b$, the expected covered fraction would be

$$1 - \left(1 - \frac{1}{N}\right)^{mt} \approx 1 - e^{-mt/N}.$$

But a Hellman table does not draw mt independent samples. It samples start points, then applies the same function F repeatedly. If two chains meet, they coalesce forever:

$$F^a(s_i) = F^b(s_j) \implies F^{a+r}(s_i) = F^{b+r}(s_j) \text{ for all } r \geq 0.$$

Thus the functional-graph structure of F can reduce actual coverage far below the independent-sampling reference.

This is not a measurement anomaly; it is a structural effect. It is also why exact coverage analysis is included in the repository rather than relying on the reference value in Eq. (11.1).

12 Distinguished points

A distinguished-point method, closely related to the collision-search framework of van Oorschot and Wiener [2], stores only chains that reach an easily tested subset $D \subset X_b$. In the modern implementation,

$$D_d = \{x \in X_b : x \bmod 2^d = 0\}.$$

Equivalently, the lowest d state bits are zero. Under a uniform-state heuristic, the expected hitting time is approximately 2^d transitions.

A chain starts at s_i and is followed until either:

1. it reaches a distinguished point; or
2. it exceeds a maximum chain length.

Only successful chains are stored. Chains that do not hit a distinguished point before the cutoff are counted as truncated. During lookup, the target is projected forward until it reaches a distinguished point or the bound is exceeded. Candidate chains sharing that endpoint are regenerated and verified against the target ciphertext.

13 Implementation corrections

The modern implementation changes the historical prototype in several ways:

1. The reduced key space counts effective DES key bits, not raw bytes.
2. Endpoints are indexed rather than searched by nested linear scans.
3. Candidate chains are always verified against the target ciphertext.
4. Distinguished-point chains are bounded and truncated chains are counted.
5. The implementation records exact state coverage outside the timed path.
6. All experiments are generated from explicit seeds.
7. A separate historical-math verifier checks the slide-7 $\mathbb{F}(16)$ multiplication formula, the slide-8 squaring permutation, and $DD^{-1} = I_8$ for the slide-12 matrices.

These changes make the code a reference implementation rather than a one-off demonstration. They also separate algorithmic claims from JVM timing noise.

14 Verified experiment

The committed benchmark uses:

- effective state dimension $b = 16$, so $N = 65\,536$;
- 30 deterministic seeded trials;
- Hellman parameters $m = 1024$, $t = 64$;
- distinguished-point parameters 1024 starts, $d = 6$, and maximum chain length 256;
- a separate warm-up run before collecting timings;
- exact unique-state table coverage computed outside the timed path.

Method	Exact recovery	Median exact coverage	Median online time
Exhaustive	30/30	100.0%	25.863 ms
Hellman	4/30	16.8%	3.399 ms
Distinguished points	3/30	15.4%	1.790 ms

For Hellman, the parameters satisfy

$$\frac{mt}{N} = \frac{1024 \cdot 64}{65\,536} = 1.$$

The independent-sample occupancy reference would therefore be

$$1 - e^{-1} \simeq 63.2\%.$$

The exact measured median coverage is instead 16.8%. This confirms that the fixed-map functional graph is the right mathematical model for these tables.

For distinguished points, the expected waiting-time reference is

$$2^d = 64$$

transitions. Across the 30 tables, the median exact stored-table coverage is 15.4%, and many starts merge onto the same distinguished endpoint.

15 Rust reference implementation and equivalence roadmap

The Java implementation remains the current reference semantics. On 28 August 2026 we added an initial Rust reference implementation rather than a performance rewrite. The Rust crate reproduces the reduced DES state encoding, PKCS#5-padded DES oracle, legacy reduction, exhaustive search, Hellman chains, distinguished-point chains, and exact coverage analysis. Shared reference vectors are consumed by both Java and Rust tests. Before any cross-language performance claim, the Rust implementation must continue to reproduce:

1. the same reduced DES state encoding;
2. the same key parity convention;
3. the same reduction function;
4. the same chain endpoints for fixed seeds;
5. the same exact coverage results;
6. the same recovery decisions on committed test vectors.

Only after cross-language equivalence is established beyond the committed reference vectors should performance work begin. The implementation roadmap is:

Java reference $\longrightarrow$ test vectors $\longrightarrow$ Rust reference (initial implementation)
 $\longrightarrow$ equivalence tests $\longrightarrow$ parallel/SIMD optimization.

The committed Rust crate deliberately has no performance claim yet. Future work includes property-based testing, Criterion-style benchmarking, larger cross-language equivalence suites, and parallel chain generation. A future hardware path would treat encryption and reduction as pipeline kernels, but that is outside the current repository.

16 Future research directions

Several directions follow naturally from this update.

Multiple reductions. Classical Hellman tables often use table-dependent reductions. The current implementation uses one fixed reduction. Introducing indexed reductions R_j would reduce chain merging and make the comparison with classical tradeoff theory more direct.

Rainbow and multi-table variants. Rainbow tables change the reduction per column. A reproducible comparison between fixed-reduction Hellman tables, multiple Hellman tables, and rainbow tables would clarify how much of the measured coverage loss is due to functional-graph coalescence.

Parameter optimization. The current benchmark is deliberately small. A larger study should sweep m , t , distinguished-point difficulty d , and chain bounds, reporting coverage, online work, false candidates, and memory.

Finite-field arithmetic implementation. The AES finite-field part should be turned into executable code: basis-change matrices, composite-field inversion, δ multiplication, and non-LUT S-box circuits. The code should report XOR count, depth, and test equivalence against the AES S-box.

Rust and hardware. Rust should first match Java semantics and only then pursue speed. Hardware work should be based on stable test vectors and should separate AES arithmetic optimization from time-memory tradeoff table generation.

17 Conclusion

The updated repository separates historical provenance from modern reproducibility. The original presentation remains intact, while this note reconstructs the finite-field mathematics and connects it to a tested time-memory tradeoff implementation.

The most important experimental lesson is that Hellman table size alone does not determine coverage. For the tested fixed reduction, $mt/N = 1$ but exact coverage is only about 16.8%. The correct model is the functional graph of $F(x) = R(\text{Enc}_{K(x)}(P))$, where chain coalescence is structural. This makes the project a useful base for a verified Rust implementation and for a broader study of reductions, table families, and cryptographic datapath representations.

References

- [1] M. E. Hellman. A cryptanalytic time-memory trade-off. *IEEE Transactions on Information Theory*, 26(4):401–406, 1980.
- [2] P. C. van Oorschot and M. J. Wiener. Parallel collision search with cryptanalytic applications. *Journal of Cryptology*, 12:1–28, 1999.
- [3] National Institute of Standards and Technology. *Advanced Encryption Standard (AES)*, FIPS 197, updated 2023.
- [4] J. Daemen and V. Rijmen. *The Design of Rijndael: AES – The Advanced Encryption Standard*. Springer, 2002.

- [5] D. Canright. A very compact S-box for AES. In *Cryptographic Hardware and Embedded Systems – CHES 2005*, pages 441–455. Springer, 2005.
- [6] A. Satoh, S. Morioka, K. Takano, and S. Munetoh. A compact Rijndael hardware architecture with S-box optimization. In *Advances in Cryptology – ASIACRYPT 2001*, pages 239–254. Springer, 2001.
- [7] T. Itoh and S. Tsujii. A fast algorithm for computing multiplicative inverses in $\text{GF}(2^m)$ using normal bases. *Information and Computation*, 78(3):171–177, 1988. DOI: 10.1016/0890-5401(88)90024-7.
- [8] A. Sakho, A. Mkhida, and M. El Yadari. *High Performance Computing for Cryptography*. M2P SCCI, Ensimag–UJF, historical master’s research presentation, undated.